```
%================================================================
% Deterministic Cake-Eating Problem: log utility
%================================================================

clear
clc

%Printing the outputs as a text file if ==1
output_file=0;

%LaTeX Interpreter if ==1 (requires LaTeX to be installed if compiled on Octave)
latex_interpreter=1;

%----------------------------------------------------------------
% Grid over Cake Size
%----------------------------------------------------------------
dimW=2000;                    %Number of nodes
gridW=linspace(0.001,1,dimW);  %Uniform grid over cake size
% It starts with 0.001 due to log(0)=-Inf.

%----------------------------------------------------------------
% Defining the Auxiliary Matrix of Consumption
% for Each Combination of Today and Tommorow's States
%----------------------------------------------------------------
% Consumption can be recovered from current and next period states.
% Creating the (dimW,dimW)-matrix such that an (i,j) element is
% the amount of consumption associated with
% w=gridW(i) (current state is the i-th node) and
% w_prime=gridW(j) (next period's state is the j-th node).
% Note that a choice variable is the next-period state.
consumption = gridW'*ones(1,dimW)-ones(dimW,1)*gridW;
consumption(consumption<=0)=0.1^16;
% gridW'*ones(1,dimW) yields the (dimW,dimW) matrix ( gridW' ... gridW' )
% ones(dimW,1)*gridW yields the (dimW,dimW) matrix ( gridW; ... gridW )
% Replace non-positive (infeasible) consumption with
% something sufficiently close to zero.

%----------------------------------------------------------------
% Setting Parametric Values for the Value Function Iteration
%----------------------------------------------------------------
beta=0.85;       %Discount Factor
max_iter=1000;   %The Maximum Number of Iterations
toler=0.1^16;    %Tolerance Value
converge=0;      %An optional binary variable
% converge takes 1 when the value-function iteration ended
% within the maximum number of rounds (max_iter).

%----------------------------------------------------------------
% Initialize the Value Function (Initial Guess)
%----------------------------------------------------------------
% V is a (1, dimW)-matrix with (1,i)-th element denoting the value
% (of a candidate value function) associated with state being i (i.e., gridW(i)).
% Here, simply start V with the vector of zero.
V=zeros(1,dimW);

%----------------------------------------------------------------
% Value function Iteration
%----------------------------------------------------------------
```

```
tic %start stopwatch timer (optional)
for i=1:max_iter
    [newV,index]=max( (log(consumption) + beta*ones(dimW,1)*V)' );
    % The (i,j) element of the matrix (log(consumption) + beta*ones(dimW,1)*V)
    % is the utility associated with state i and choice j
    % max(A') is a row vector containing the maximum value of each row of A
    % newV, a (1,dimW)-matrix, is the new/updated value function
    % index is a (1,dimW)-matrix with (1,i) element denoting
    % optimal choice ("optimal j") for each current state i

    %Alternative
    %[newV,index]=max( (log(consumption) + beta*ones(dimW,1)*V),[],2 );
    %newV=newV';

    if norm(newV-V,Inf)<toler %alternative: if max(abs(newV-V))<toler
        converge=1;
        break;
    end %end of if
    V=newV;
end %end of for
num_policy=gridW-gridW(index); %Policy function (c=w-w')
comp_time=toc; %end stopwatch timer and substitute time into comp_time (optional)

%-----------------------------------------------------------------
% Checking Whether the Value Function Converged (Optional)
%-----------------------------------------------------------------
if output_file==1
    dfile="DP_cake_eating.txt";
    if exist(dfile, 'file')
        delete(dfile);
    end
    diary(dfile)
    diary on
end

if converge==1
    fprintf('The value function iteration converged with %d iterations.\n', i);
    fprintf('The value function iteration took %f seconds.\n', comp_time);
else
    fprintf('The value function iteration did not converge.\n');
end

if output_file==1
    diary off
end

%-----------------------------------------------------------------
% Plotting Policy Function
%-----------------------------------------------------------------
% Plot analytical and numerical solutions to the policy function
% Analytical solution: h(w)=(1-beta)*w
exact_policy=(1-beta)*gridW;

figure(1)
plot(gridW,[exact_policy;num_policy],'LineWidth',1);
grid on
axis([0 1 0 (1-beta)*1.05]);
if latex_interpreter==1
```

```
    title('Optimal Policy Function','Interpreter','latex','FontSize',14);
    xlabel('Size of Cake','Interpreter','latex','FontSize',12);
    ylabel('Consumption','Interpreter','latex','FontSize',12);
    legend('Analytical', 'Numerical','Location','southeast','Interpreter','latex');
else
    title('Optimal Policy Function','FontSize',14);
    xlabel('Size of Cake','FontSize',12);
    ylabel('Consumption','FontSize',12);
    legend('Analytical', 'Numerical','Location','southeast');
end

%-----------------------------------------------------------------
% Plotting Value function
%-----------------------------------------------------------------
exact_V= ((log(1-beta)/(1-beta))+ (beta/((1-beta)^2))*log(beta))*ones(1,dimW)+ (log(gridW)./(1-
beta));
figure(2)
plot(gridW, [exact_V;V],'LineWidth',1);
grid on
%axis([0 1 min([analytical_V V]) max([analytical_V V])]);
if latex_interpreter==1
    title('Value Function','Interpreter','latex','FontSize',14);
    xlabel('Size of Cake','Interpreter','latex','FontSize',12);
    ylabel('Value','Interpreter','latex','FontSize',12);
    legend('Analytical', 'Numerical','Location','southeast','Interpreter','latex');
else
    title('Value Function','FontSize',14);
    xlabel('Size of Cake','FontSize',12);
    ylabel('Value','FontSize',12);
    legend('Analytical', 'Numerical','Location','southeast');
end

%-----------------------------------------------------------------
% Computing Optimal Consumption/State Paths
%-----------------------------------------------------------------
num_period=50; %The number of periods for computation
% num_period has to be relatively small so that the optimal consumption
% is larger than the minimum node.
% In the program, once the optimal consumption hits the minimum node then
% stays at the minimum node.
analytical_C=(1-beta)*beta.^(0:num_period-1);
analytical_W=beta.^(0:num_period-1);

Windex=zeros(1,num_period+1);
% Windex is a (1,num_period+1)-matrix with (1,i) element
% denoting state at period (i-1) on the optimum path
Windex(1,1)=dimW; %At time 0, W_0=dimW-th (initial) state
for period=1:num_period
    if Windex(1,period)>1
        Windex(1,period+1)=index(Windex(1,period));
    else
        Windex(1,period+1)=1;
    end
end
Wsim=gridW(Windex);
Csim=Wsim(1:num_period)-Wsim(2:num_period+1);

figure(3)
```

```
plot(0:num_period-1,[analytical_C;Csim],'LineWidth',1);
grid on
if latex_interpreter==1
    xlabel('Period','Interpreter','latex','FontSize',12);
    ylabel('Consumption','Interpreter','latex','FontSize',12)
    title('Optimal Consumption Path','Interpreter','latex','FontSize',14);
    legend('Analytical', 'Numerical','Interpreter','latex');
else
    xlabel('Period','FontSize',12);
    ylabel('Consumption','FontSize',12)
    title('Optimal Consumption Path','FontSize',14);
    legend('Analytical', 'Numerical');
end

figure(4)
plot(0:num_period-1,[analytical_W;Wsim(1:num_period)],'LineWidth',1);
grid on
if latex_interpreter==1
    xlabel('Period','Interpreter','latex','FontSize',12);
    ylabel('Cake Size','Interpreter','latex','FontSize',12);
    title('Optimal Path of States','Interpreter','latex','FontSize',14);
    legend('Analytical', 'Numerical','Interpreter','latex');
else
    xlabel('Period','FontSize',12);
    ylabel('Cake Size','FontSize',12);
    title('Optimal Path of States','FontSize',14);
    legend('Analytical', 'Numerical');
end

```